

云计算环境中使用虚拟机的动态资源分配

肖震, IEEE 高级会员、宋伟佳和陈琪

摘要: 云计算允许企业客户根据需要扩大和缩小资源使用量。云模型中许多被吹捧的收益来自于通过虚拟化技术实现的资源复用。在本文中, 我们提出了一种系统, 该系统使用虚拟化技术根据应用需求动态分配数据中心资源, 并通过优化使用的服务器数量来支持绿色计算。我们引入“偏度”的概念来衡量服务器多维资源利用率的不均匀性。通过最小化偏度, 我们可以很好地组合不同类型的工作负载, 提高服务器资源的整体利用率。我们开发了一套启发式方法, 可以有效防止系统过载, 同时节省能源。迹线驱动的仿真和实验结果表明我们的算法取得了良好的性能。

索引术语——云计算、资源管理、虚拟化、绿色计算



1 我NTRODUCTION

云计算所提供的弹性和缺乏前期资本投资对许多企业有吸引力。关于云模型的优点和成本以及如何将遗留应用程序迁移到云平台上有很多讨论。这里我们研究一个不同的问题: 云服务提供商如何最好地将其虚拟资源复用到物理硬件上? 这很重要, 因为云模型中的大部分被吹捧的收益都来自于这种多路复用。研究发现, 由于高峰需求的过度配置, 许多现有数据中心的服务器常常严重未得到充分利用[1]、[2]。云模型预计将通过提供自动扩展和缩减来响应负载变化, 从而使这种做法变得不必要。除了降低硬件成本外, 它还节省了电力, 这在大型数据中心的运营支出中占很大一部分。

像 Xen 这样的虚拟机监视器 (VMM) 提供了一种将虚拟机 (VM) 映射到物理资源的机制 [3]。这种映射在很大程度上对云用户是隐藏的。例如, 使用 Amazon EC2 服务 [4] 的用户不知道他们的 VM 实例在哪里运行。云提供商有责任确保底层物理机 (PM) 有足够的资源来满足其需求。VM 实时迁移技术使得在应用程序运行时改变 VM 和 PM 之间的映射成为可能 [5]、[6]。然而, 如何自适应地决定映射以便在满足 VM 的资源需求的同时最小化所使用的 PM 的数量仍然是一个策略问题。这是

当虚拟机的资源需求是异构的时, 由于它们运行不同的应用程序集, 并且随着工作负载的增长和收缩而变化, 因此资源需求是异构的, 这是一个挑战。The capacity of PMs can also be heterogenous because multiple generations of hardware coexist in a data center.

我们的算法旨在实现两个目标:

。避免过载。PM 的容量应足以满足其上运行的所有 VM 的资源需求。否则, PM 会过载并可能导致其 VM 性能下降。绿色计算。只要 PM 仍能满足所有 VM 的需求, 应尽量减少使用的 PM 数量。可以关闭空闲的 PM 以节省能源。

面对不断变化的虚拟机资源需求, 这两个目标之间存在固有的权衡。为了避免过载, 我们应该保持 PM 的利用率较低, 以减少以后 VM 的资源需求增加时发生过载的可能性。对于绿色计算, 我们应该保持 PM 的利用率较高, 以有效利用其能源。

在本文中, 我们介绍了自动化资源管理系统的设计和实现, 该系统在两个目标之间实现了良好的平衡。我们做出以下贡献:

。我们开发了一个资源分配系统, 可以有效避免系统过载, 同时最大限度地减少服务器的使用数量。我们引入“偏度”的概念来衡量服务器利用率的不均匀程度。通过最小化偏度, 我们可以在面对多维资源约束的情况下提高服务器的整体利用率。我们设计了一种负载预测算法, 可以准确捕获应用程序的未来资源使用情况, 而无需查看虚拟机内部。该算法可以捕捉资源使用模式的上升趋势, 并有助于显着减少布局流失。

• The authors are with the Department of Computer Science, Peking University, No. 5, Yiheyuan Road, Haidian District, Beijing 100871, P.R. China. E-mail: {xiaozhen, songweijia}@pku.edu.cn, chenqi@net.pku.edu.cn.

Manuscript received 26 Feb. 2012; revised 18 July 2012; accepted 17 Sept. 2012; published online 21 Sept. 2012.

Recommended for acceptance by V.B. Misic, R. Buyya, D. Milojicic, and Y. Cui.

For information on obtaining reprints of this article, please send e-mail to: tpds@computer.org, and reference IEEECS Log Number TPDSSI-2012-02-0148.

Digital Object Identifier no. 10.1109/TPDS.2012.283.

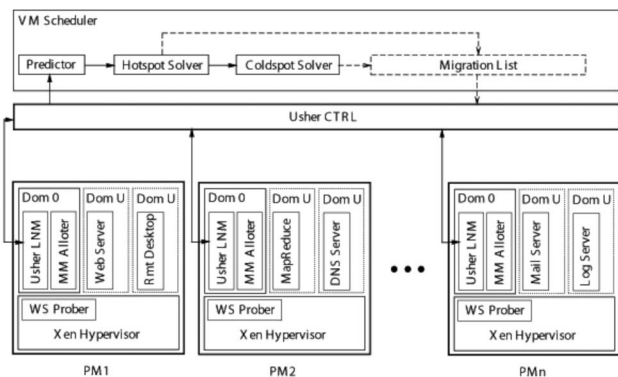


图 1. 系统架构。

本文的其余部分组织如下。第 2 节概述了我们的系统，第 3 节描述了我们预测资源使用情况的算法。我们的算法的细节在第 4 节中介绍。第 5 节和第 6 节分别介绍了模拟和实验结果。第 7 节讨论相关工作。第 8 节总结。

2 SYSTEM OVERVIEW

该系统的架构如图1所示。每个PM运行Xen管理程序（VMM），它支持特权域0和一个或多个域U [3]。域U中的每个VM都封装了一个或多个应用程序，例如Web服务器、远程桌面、DNS、Mail、Map/Reduce等。我们假设所有PM共享一个后端存储。

VM 到 PM 的复用是使用 Usher 框架 [7] 进行管理。我们系统的主要逻辑是作为 Usher 的一组插件实现的。每个节点在域 0 上运行 Usher 本地节点管理器 (LNM)，该管理器收集该节点上每个虚拟机的资源使用统计信息。通过监控Xen中的调度事件可以计算出CPU和网络的使用情况。然而，虚拟机内的内存使用情况对于虚拟机管理程序来说是不可见的。一种方法是通过观察虚拟机的交换活动来推断虚拟机的内存不足[8]。不幸的是，客户操作系统需要安装单独的交换分区。此外，在交换发生时调整内存分配可能为时已晚。相反，我们在每个虚拟机管理程序上实现了一个工作集探测器（WS Prober），以估计其上运行的虚拟机的工作集大小。我们使用VMware ESX Server [9] 中的随机页面采样技术。

每个 PM 收集的统计信息将转发到运行 VM 调度程序的 Usher 中央控制器 (Usher CTRL)。VM Scheduler定期被调用，并从LNM接收VM的资源需求历史、PM的容量和负载历史以及VM在PM上的当前布局。

调度程序有几个组件。预测器根据过去的统计数据预测VM未来的资源需求和PM的未来负载。我们通过聚合 VM 的资源使用情况来计算 PM 的负载。负载预测算法的细节将在下一节中描述。每个节点的LNM首先尝试通过调整共享同一VMM的VM的资源分配来满足本地的新需求。Xen

可以通过调整 CPU 调度程序中的权重来更改 VM 之间的 CPU 分配。每个节点的domain 0上的MM Alloter负责调整本地内存分配。

VM Scheduler 中的热点求解器会检测任何 PM 的资源利用率是否高于热点阈值（即热点）。如果是这样，运行在其上的一些虚拟机将被迁移以减少负载。冷点求解器检查活跃使用的 PM (APM) 的平均利用率是否低于绿色计算阈值。如果是这样，其中一些 PM 可能会被关闭以节省能源。它识别利用率低于冷阈值（即冷点）的 PM 集，然后尝试迁移掉其所有 VM。然后，它编译 VM 迁移列表并将其传递给 Usher CTRL 执行。

3 PREDICTING FUTURE RESOURCE NEEDS

我们需要预测虚拟机未来的资源需求。正如前面所说，我们的重点是互联网应用。一种解决方案是查看虚拟机内部的应用程序级别统计信息，例如通过解析待处理请求的日志。这样做需要修改虚拟机，但这并不总是可行。相反，我们根据虚拟机过去的外部行为进行预测。我们的第一次尝试是使用类似 TCP 的方案来计算指数加权移动平均线 (EWMA)

$$E(t) = \alpha * E(t-1) + (1 - \alpha) * O(t), 0 \leq \alpha \leq 1,$$

其中 $E(t)$ 和 $O(t)$ 分别是时间 t 时的估计负载和观测负载。 α 反映了稳定性和响应能力之间的权衡。

我们使用 EWMA 公式来预测我们大学 DNS 服务器上的 CPU 负载。我们每分钟测量负载并预测下一分钟的负载。图 2 a 显示了 $\alpha = 0.7$ 的结果。图中的每个点都是观测值，曲线代表预测值。从视觉上看，曲线穿过点的中间，这表明预测相当准确。表1中的统计数据也验证了这一点。括号中的参数是 α 值。W 是测量窗口的长度（稍后解释）。“中值”误差以观测值的百分比计算： $(E(t) - O(t)) / O(t)$ 。“较高”和“较低”误差百分比分别是预测值高于或低于观测值的百分比。正如我们所看到的，预测相当准确，较高值和较低值的百分比大致相等。

尽管看似令人满意，但该公式并未反映资源使用的上升趋势。例如，当我们看到序列 $O(t) = 10; 20; 30$ 和 40 ，预测下一个值为 50 是合理的。不幸的是，当 α 介于 0 和 1 之间时，预测值始终介于历史值和观测值之间。为了反映“加速度”，我们采取了一种创新方法，将 α 设置为负值。当 $-1 \leq \alpha < 0$ 时，上式可转化为：

$$\begin{aligned} E(t) &= -|\alpha| * E(t-1) + (1 + |\alpha|) * O(t) \\ &= O(t) + |\alpha| * (O(t) - E(t-1)), \end{aligned}$$

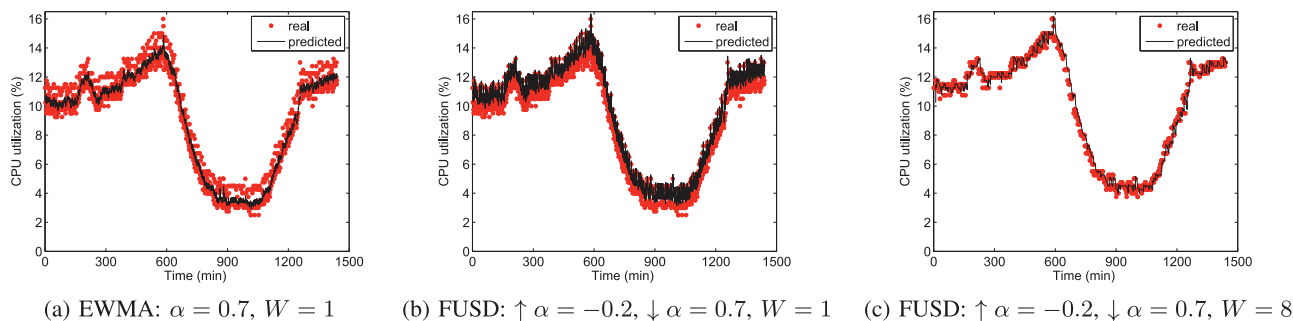


图 2. 我们大学 DNS 服务器的 CPU 负载预测。W 是测量窗口。

另一方面，当观察到的资源使用量下降时，我们希望保守地减少我们的估计。因此，我们使用两个参数“ α 和 $\# \alpha$ ”来分别控制当 $O_{\delta t}$ 增加或减少时 $E_{\delta t}$ 适应变化的速度。我们称之为快速上升和减速 (FUSD) 算法。图 2b 显示了 FUSD 算法对于“ $\alpha = -0.2$ 、 $\# \alpha = 0.7$ ”的有效性。（这些值是根据对多个互联网应用程序收集的跟踪的现场经验选择的。）现在，大多数时候预测值都高于观察值：根据表 1，为 77%。中值误差增加到 9.4%，因为我们以准确性换取安全性。尽管如此，这仍然是可以接受的。

到目前为止，我们将 $O_{\delta t}$ 作为最后观察到的值。大多数应用程序都根据满足特定性能级别的请求的特定百分位数来指定其 SLO。更一般地说，我们保留最近观察到的 W 值的窗口，并将 $O_{\delta t}$ 作为其中的高百分位数。图 2c 显示了当 $W = 1/48$ 时的结果，我们取峰值资源需求的第 90% 个百分位数。该图显示预测结果明显改善。

我们还研究了其他预测算法。例如，线性自回归 (AR) 模型在其他工作[10]、[11]、[12]的负载预测中被广泛采用。它将预测值建模为其过去观察值的线性函数。模型参数是通过使用历史值进行训练来确定的。AR 预测器能够纳入负荷变化的季节性模式。例如，SPAR(4,2) [10] 根据过去的 6 次观测来估计 MSN 客户端的未来记录率，其中 2 次是最新观测，另外 4 次是过去 4 周内同一时间的观测。

我们比较 SPAR(4,2) 和 FUSD $\delta=0.2$ ；图 3 中的 0.7p。“lpct”指的是低误差百分比，而“std”指的是标准偏差。这两种算法都用于预测上述 DNS 服务器在 1 天内的 CPU 利用率。预测窗口为

表 1 负载预测算法

	ewma(0.7) $W = 1$	fusd(-0.2, 0.7) $W = 1$	fusd(-0.2, 0.7) $W = 8$
median error	5.6%	9.4%	3.3%
high error	56%	77%	58%
low error	44%	23%	41%

8分钟。SPAR (4,2) 的标准差 (std) 比 FUSD $\delta=0.2$ 小约 16%；0.7p，这意味着 SPAR (4,2) 实现了明显更好的精度。这是因为它利用了负载的潮汐模式。然而，SPAR(4,2) 既不能避免低预测，也不能平滑负载。需要训练阶段来确定参数是不方便的，特别是当负载模式发生变化时。因此，我们采用更简单的 EWMA 方差。对预测算法的彻底研究留待未来的工作。

正如我们将在本文后面看到的，预测算法在提高资源分配决策的稳定性和性能方面发挥着重要作用。

4 THE SKEWNESS ALGORITHM

我们引入偏度的概念来量化服务器上多个资源利用的不均匀性。令 n 为我们考虑的资源数量， r_i 为第 i 个资源的利用率。我们将服务器的资源偏度定义为

$$skewness(p) = \sqrt{\sum_{i=1}^n \left(\frac{r_i}{\bar{r}} - 1 \right)^2},$$

其中 $\bar{r}$ 是服务器 p 所有资源的平均利用率。实际上，并非所有类型的资源都对性能至关重要，因此我们在上述计算中只需要考虑瓶颈资源。通过最小化偏度，我们可以很好地组合不同类型的工作负载，提高服务器资源的整体利用率。下面，我们描述算法的细节。算法的分析在补充文件的第 1 节中介绍，可以是

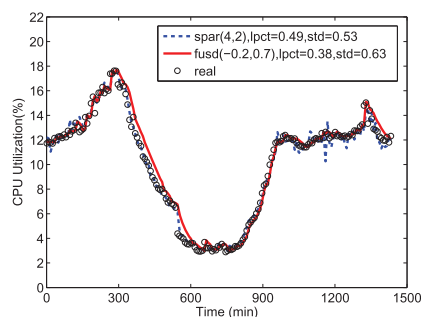


图 3. SPAR 和 FUSD 的比较。

可在 ComputerSociety 数字图书馆 <http://doi.ieeecomputersociety.org/10.1109/TPDS.2012.283> 上找到。

4.1 热点和冷点

我们的算法定期执行, 根据预测的虚拟机未来资源需求来评估资源分配状态。如果服务器任何资源的利用率高于热点阈值, 我们将服务器定义为热点。这表明服务器过载, 因此应迁移掉其上运行的一些虚拟机。我们将热点 p 的温度定义为超出热点阈值的资源利用率的平方和:

$$temperature(p) = \sum_{r \in R} (r - r_t)^2,$$

其中 R 是服务器 p 中过载资源的集合, r_t 是资源 r 的热阈值。(注意, 计算中仅考虑过载的资源。) 热点的温度反映了其过载的程度。如果服务器不是热点, 则其温度为零。

We define a server as a cold spot if the utilizations of all its resources are below a cold threshold. This indicates that the server is mostly idle and a potential candidate to turn off to save energy. 然而, 只有当系统中所有活跃使用的服务器 (即 APM) 的平均资源利用率低于绿色计算阈值时, 我们才会这样做。A server is actively used if it has at least one VM running. 否则, 它是不活动的。最后, 我们将热阈值定义为资源利用率的水平, 该水平足够高以证明服务器运行是合理的, 但又不能高到在面临应用程序资源需求临时波动时成为热点的风险。

不同类型的资源可以有不同的阈值。例如, 我们可以将 CPU 和内存资源的热阈值分别定义为 90% 和 80%。因此, 如果服务器的 CPU 使用率高于 90% 或内存使用率高于 80%, 则该服务器就是热点。

4.2 热点缓解

我们按照温度降序对系统中的热点列表进行排序 (即, 我们首先处理最热的热点)。我们的目标是尽可能消除所有热点。否则, 请保持尽可能低的温度。对于每个服务器 p , 我们首先决定应迁移掉其哪些虚拟机。如果虚拟机被迁移, 我们会根据服务器的最终温度对其虚拟机列表进行排序。我们的目标是迁移掉最能降低服务器温度的虚拟机。如果出现平局, 我们选择删除最能减少服务器偏度的虚拟机。对于列表中的每个虚拟机, 我们查看是否可以找到一个目标服务器来容纳它。接受此虚拟机后, 服务器不得成为热点。在所有这样的服务器中, 我们选择一个通过接受这个虚拟机可以最大程度地降低偏度的服务器。请注意, 这种减少可能是负数, 这意味着我们选择偏度增加最少的服务器。如果找到目标服务器, 我们会记录虚拟机到该服务器的迁移并更新相关服务器的预测负载。否则, 我们将移至列表中的下一个虚拟机并尝试为其找到目标服务器。只要我们能有任何虚拟机找到目标服务器, 我们就认为算法的这次运行成功, 然后

移动到下一个热点。请注意, 每次运行该算法都会从过载的服务器中迁移最多一台虚拟机。这不一定消除热点, 但至少降低其温度。如果它在下一次决策运行中仍然是热点, 算法将重复此过程。可以设计算法, 使其可以在每次运行期间迁移多个虚拟机。但这可能会在相关服务器已经过载的时期增加更多负载。我们决定使用这种更保守的方法, 并在启动其他迁移之前给系统一些反应时间。

4.3 绿色计算

当活动服务器的资源利用率太低时, 可以关闭其中一些服务器以节省能源。这是在我们的绿色计算算法中处理的。这里的挑战是在不牺牲现在或将来性能的情况下减少低负载期间活动服务器的数量。我们需要避免系统振荡。

当活动服务器上所有资源的平均利用率低于绿色计算阈值时, 就会调用我们的绿色计算算法。我们根据内存大小的升序对系统中的冷点列表进行排序。由于我们需要先迁移掉其所有虚拟机, 然后才能关闭未充分利用的服务器, 因此我们将冷点的内存大小定义为其上运行的所有虚拟机的内存总大小。回想一下, 我们的模型假设所有虚拟机都连接到共享后端存储。因此, 虚拟机实时迁移的成本主要取决于其内存占用。补充文件中的第 7 节解释了为什么记忆是一个很好的深度衡量标准。我们首先尝试以最低的成本消除冷点。

对于冷点 p , 我们检查是否可以将其所有虚拟机迁移到其他地方。对于 p 上的每个虚拟机, 我们尝试找到一个目标服务器来容纳它。接受虚拟机后服务器的资源利用率必须低于热阈值。虽然我们可以通过整合未充分利用的服务器来节省能源, 但过度整合可能会在未来产生热点。温暖阈值旨在防止这种情况发生。如果多个服务器满足上述标准, 我们会优先选择当前不是冷点的服务器。这是因为增加冷点上的负载会降低其被消除的可能性。但是, 如有必要, 我们将接受冷点作为目标服务器。在所有条件相同的情况下, 我们选择一个目标服务器, 通过接受该虚拟机, 其偏差可以最大程度地降低。如果我们能够找到冷点上所有虚拟机的目标服务器, 我们会记录迁移顺序并更新相关服务器的预测负载。否则, 我们不会迁移其任何虚拟机。冷点列表也会更新, 因为由于上述过程中建议的虚拟机迁移, 其中一些可能不再是冷点。

上述整合增加了相关服务器的额外负载。这并不像热点缓解情况那样严重, 因为绿色计算仅在系统负载较低时启动。尽管如此, 我们希望限制服务器整合带来的额外负载。我们限制每次运行算法时可以消除的冷点数量不超过系统中活动服务器的一定百分比。这称为合并限制。

表 2 我们的模拟中的参数

symbol	meaning	value
h	hot threshold	0.9
c	cold threshold	0.25
w	warm threshold	0.65
g	green computing threshold	0.4
l	consolidation limit	0.05

请注意，仅当所有活动服务器 (APM) 的平均负载低于绿色计算阈值时，我们才会消除系统中的冷点。否则，我们会将这些冷点留在那里作为未来卸载的潜在目标机器。这与我们的绿色计算应该保守进行的理念是一致的。

4.4 合并运动

上述每个步骤中生成的运动只有在所有步骤完成后才会执行。然后合并移动列表，以便每个虚拟机最多移动一次到其最终目的地。例如，热点缓解可能要求VM从PM A移动到PM B，而绿色计算则要求VM从PM B移动到PM C。在实际执行中，VM直接从A移动到C。

5 SIMULATIONS

我们使用跟踪驱动模拟来评估算法的性能。请注意，我们的模拟使用与实验中实际实现相同的算法代码库。这确保了我们的模拟结果的保真度。跟踪是每分钟的服务器资源利用率，例如 CPU 速率、内存使用情况和网络流量统计数据，使用“perfmon” (Windows)、`“/proc”` 文件系统(Linux)、`“pmstat/vmstat/netstat”` 命令(Solaris) 等工具收集。原始跟踪被预处理为“Usher”格式，以便模拟器可以读取它们。我们从各种来源收集了痕迹：

。网络信息商城。中国最大的在线网络档案馆（相当于美国的互联网档案馆），拥有超过三十亿个存档网页。。真实课程。中国最大的在线远程学习系统，服务器分布在13个主要城市。。惊人的商店。国内最大的P2P存储系统。

我们还从我们大学的服务器和桌面计算机收集了痕迹，包括我们的邮件服务器之一、中央 DNS 服务器和我们部门的桌面。我们根据收集的天数对跟踪进行后处理，并使用随机采样和数据集的线性组合来生成所需的工作负载。除非另有说明，本节中的所有模拟均使用真实跟踪工作负载。

我们在模拟中使用的默认参数如表 2 所示。我们使用了 F USD 负载预测算法，其中 $\alpha_{1/4} = 0.2$ 、 $\beta_{1/4} = 0.7$ 和 $W_{1/4} = 8$ 。在动态系统中，这些参数代表了自适应调整系统性能的良好旋钮。我们根据经验选择默认参数值

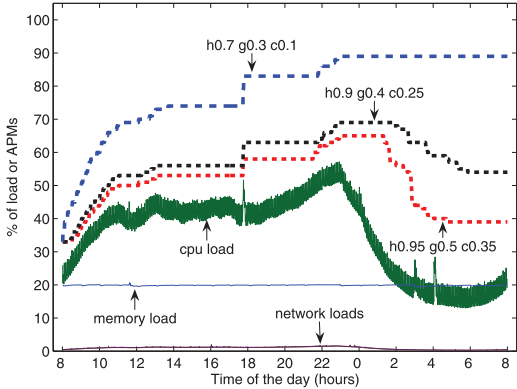


图 4. 阈值对 APM 数量的影响。

与许多互联网应用程序一起工作。未来，我们计划探索使用人工智能或控制理论方法来自动找到接近最优值。

5.1 阈值对 APM 的影响

我们首先评估算法中使用的各种阈值的效果。我们模拟一个具有 100 个 PM 和 1,000 个 VM（从跟踪中随机选择）的系统。我们在初始布局中使用随机 VM 到 PM 映射。调度程序每分钟调用一次。图 4 的底部显示了系统中每日的负载变化。x 轴是一天中从上午 8 点开始的时间。y 轴过载有两个含义：系统中负载的百分比或 APM（即活动 PM）的百分比。回想一下，如果 PM 至少有一个正在运行的 VM，则它是活动的（即 APM）。从图中可以看出，CPU 负载呈现昼夜模式，在午夜之后大幅下降。随着时间的推移，内存消耗相当稳定。网络利用率仍然很低。

图 4 的顶部显示了我们算法中不同阈值的 APM 百分比如何随负载变化。例如，“h0.7 g0.3 c0.1”表示热计算、绿色计算和冷计算阈值分别为70%、30%和10%。图中未显示的参数采用表 2 中的默认值。通过调整阈值，我们的算法可以在迁移决策中或多或少地激进。该图显示，较低的热阈值会导致更积极的迁移，以缓解系统中的热点并增加 APM 的数量，而较高的冷和绿色计算阈值会导致更积极的整合，从而导致 APM 数量减少。使用表 2 中的默认阈值，我们算法中的 APM 百分比密切遵循负载模式。

为了检查我们的算法在更极端情况下的性能，我们还创建了一个合成工作负载，它模仿正弦函数的形状（仅正部分），范围从 15% 到 95%，具有 20% 的随机波动。它的峰均比比真实迹线大得多。结果显示在补充文件的第 2 节中，该文件可以在计算机协会数字图书馆中找到。

5.2 算法的可扩展性

我们通过在 200 到 1,400 之间改变模拟中的虚拟机数量来评估算法的可扩展性。VM与PM的比例为10:1。结果显示在

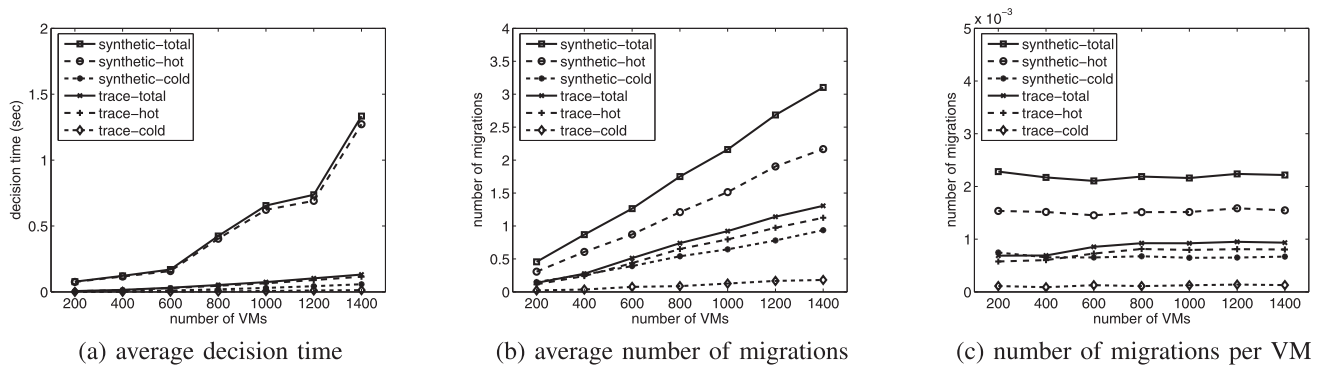


图 5. 算法随系统规模的可扩展性。

图 5。图 5a 显示我们算法的平均决策时间随着系统规模的增加而增加。增加的速度介于线性和二次之间。我们将决策时间分为两部分：热点缓解（标记为“热”）和绿色计算（标记为“冷”）。我们发现热点缓解对决策时间的贡献更大。我们还发现，由于合成工作负载的变化较大，合成工作负载的决策时间高于真实跟踪的决策时间。对于 140 个 PM 和 1,400 个虚拟机，合成工作负载的决策时间约为 1.3 秒，真实跟踪的决策时间约为 0.2 秒。

图 5b 显示了每次决策期间整个系统的平均迁移次数。迁移的数量很少，并且随着系统大小大致线性增加。我们发现热点对迁移数量的贡献更大。我们还发现合成工作负载中的迁移数量高于真实跟踪中的迁移数量。对于 140 个 PM 和 1,400 个 VM，我们的算法的每次运行平均会在整个系统中产生大约 3 次合成工作负载迁移，而对于真实跟踪仅产生 1.3 次迁移。图 5c 也验证了这一点，该图计算每个决策中每个虚拟机的平均迁移数。该图表明，每个虚拟机在决策运行期间都会经历少量、大致恒定的迁移次数，与系统大小无关。对于合成工作负载，该数字约为 0.0022；对于真实跟踪，该数字约为 0.0009。这大约相当于每 456 或 1,174 个决策间隔分别进行一次迁移。我们算法的稳定性非常好。

我们还通过改变 VM 与 PM 的比率来进行模拟。VM 与 PM 的比率越高，负载在 PM 之间的分配就越均匀。结果显示在补充文件的第 4 节中，该文件可在线获取。

5.3 负荷预测的效果

我们在图 6 中比较了有负载预测和没有负载预测的算法的执行情况。禁用负载预测时，算法在决策中仅使用最后观察到的负载。图 6a 显示，负载预测显着减少了决策运行期间系统中热点的平均数量。值得注意的是，在 1,400 个虚拟机的模拟中，预测可以防止超过 46% 的热点。这证明了其在主动防止服务器过载方面的高效性。如果没有预测，算法会在 PM 的负载降至阈值以下时立即尝试对其进行合并。通过预测，算法正确地预见到 PM 的负载将很快增加到阈值以上，因此不采取任何操作。这让总理暂时处于“冷点”状态。然而，它还通过避免由于临时负载波动而导致不必要的迁移来减少布局流失。因此，有负载预测的系统中的迁移次数比没有负载预测的系统中的迁移次数要少，如图 6c 所示。我们可以通过调整负载预测的参数来调整负载预测的保守性，但当前的配置很大程度上满足了我们的目的（即，谨慎方面的错误）。系统中存在更多冷点的唯一缺点是可能会增加 APM 的数量。图 6b 对此进行了研究，显示 APM 的平均数量与

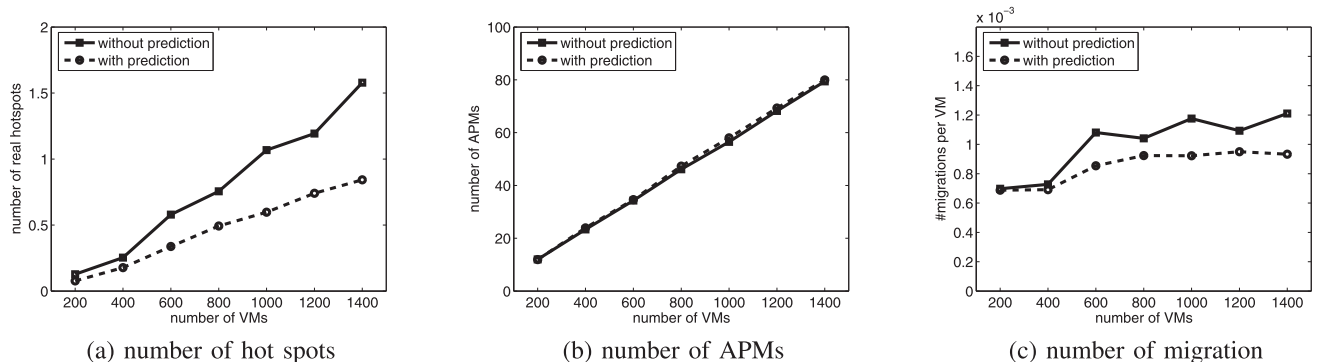


图 6. 负载预测的效果。

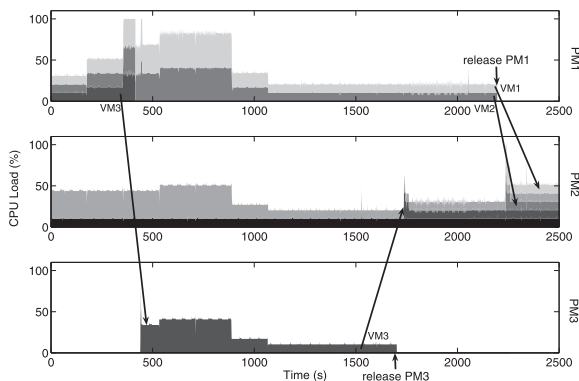


图 7.算法有效性。

或没有负载预测（差异小于 1%）。这很有吸引力，因为可以在不牺牲资源效率的情况下实现显著的过载保护。图 6c 比较了有负载预测和没有负载预测的每个决策中每个虚拟机的平均迁移数。结果表明，通过负载预测，每个虚拟机的迁移次数减少了 17%。

6 EXPERIMENTS

我们的实验是使用一组 30 台配备 Intel E5620 CPU 和 24 GB RAM 的 Dell PowerEdge 刀片服务器进行的。The servers run Xen-3.3 and Linux 2.6.18.我们使用 xenstat 库定期读取负载统计信息（与 xentop 的做法相同）。这些服务器通过千兆位以太网连接到一组四个 NFS 存储服务器，我们的 VM 调度程序在其中运行。我们使用与模拟中相同的默认参数。

6.1 算法有效性

我们评估我们的算法在减轻过载和绿色计算方面的有效性。我们从由三个 PM 和五个 VM 组成的小规模实验开始，以便我们可以在图 7 中呈现所有服务器的结果。每个 VM 使用不同的阴影。所有 VM 均配置有 128 MB RAM。每个虚拟机上都运行一个 Apache 服务器。我们使用 httpperf 在 Apache 服务器上调用 CPU 密集型 PHP 脚本。这使我们能够通过调整客户端请求率来使虚拟机承受不同程度的 CPU 负载。其他资源的利用率保持在较低水平。

我们首先增加 PM₁ 上三个虚拟机的 CPU 负载以产生过载。我们的算法通过将 VM₃ 迁移到 PM₃ 来解决过载问题。高负载下约 420 秒达到稳定状态。在 890 秒左右，我们逐渐降低所有虚拟机的 CPU 负载。由于 FUSD 预测算法在负载降低时较为保守，因此绿色计算需要一段时间才能生效。大约 1,700 秒后，VM₃ 从 PM₃ 迁移到 PM₂，以便 PM₃ 进入待机模式。大约 2,200 秒后，PM₁ 上的两个虚拟机迁移到 PM₂，以便 PM₁ 也可以释放。随着负载的上升和下降，我们的算法将重复上述过程：根据需要分散或合并虚拟机。

接下来我们将实验规模扩大到 30 台服务器。我们使用 TPC-W 基准进行此实验。TPC-W 是电子商务的行业标准基准

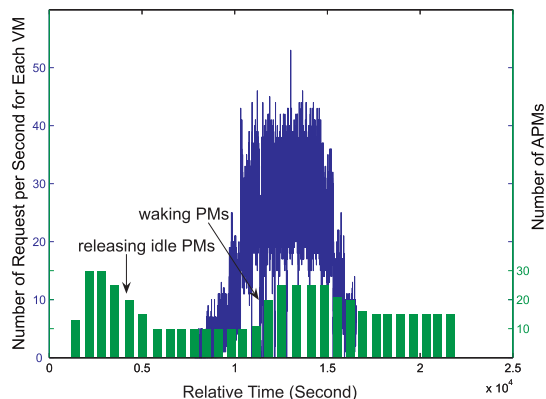


图 8.#APM 随 TPC-W 负载变化。

模拟客户浏览和购买行为的应用程序[13]。我们一开始在每台服务器上部署 8 个 VM。每个 VM 配置 1 个虚拟 CPU 和 2GB 内存。启用自我膨胀以允许管理程序回收未使用的内存。每个虚拟机都运行与各种类型的工作负载相对应的 TPC-W 基准测试的服务器端：浏览、购物、混合作业负载等。我们的算法每 10 分钟调用一次。

图 8 显示了随着时间的推移，APM 的数量如何随每个虚拟机的平均请求数量变化。我们一开始就将每个虚拟机的负载保持在较低水平。结果，绿色计算生效，并将虚拟机整合到更少数量的服务器上。¹请注意，每个 TPC-W 服务器即使在空闲时也会消耗数百兆内存。两个小时后，我们大幅增加负载以模拟“闪电人群”事件。该算法唤醒备用服务器以卸载热点服务器。从图中可以看出，APM 的数量相应增加。在请求率达到峰值约一小时后，我们逐渐减少负载以模拟闪存人群结束。这再次触发绿色计算来整合未充分利用的服务器。图 8 显示，在实验过程中，APM 数量的上升速度远远快于下降速度。这是由于我们的 FUSD 负载预测的影响。该图还显示，在闪现人群之后，APM 数量仍保持在略高的水平。这是因为 TPC-W 服务器在缓存中保留一些数据，因此其内存使用量永远不会回到原来的水平。

为了量化节能效果，我们使用刀片系统中的内置瓦特表测量了各种 TPC-W 工作负载下的电力消耗。我们发现闲置的刀片服务器消耗约 130 瓦，充分利用的服务器消耗约 205 瓦。在上述实验中，由于绿色计算，服务器平均有 48% 的时间处于待机模式。这意味着每台服务器大约节能 62 瓦，实验中使用的 30 台服务器组大约节能 1,860 瓦。

6.2 实时迁移的影响

使用虚拟机实时迁移的一个担忧是它对应用程序性能的影响。之前的研究有

1. There is a spike on the number of APMs at the very beginning because it takes a while to deploy the 240 VMs onto 30 servers.

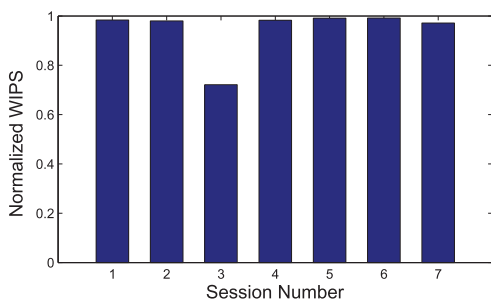


图 9. 实时迁移对 TPC-W 性能的影响。

发现这种影响很小[5]。我们在自己的实验中研究了这种影响。我们在上面的 30 个服务器实验中提取了 340 个实时迁移的数据。我们发现其中 139 个用于热点缓解。我们关注这些迁移，因为这是对应用程序性能的潜在影响最大的时候。在 139 个迁移中，我们随机挑选了 7 个相应的 TPC-W 会话进行实时迁移。所有这些会话都使用 200 个模拟浏览器运行“购物组合”工作负载。作为比较的目标，我们使用相同的参数重新运行会话，但不执行迁移，并使用结果性能作为基线。图 9 显示了 7 个会话的标准化每秒 Web 交互次数 (WIPS)。WIPS 是 TPC-W 使用的性能指标。该图显示，与基线相比，大多数实时迁移会话没有表现出明显的性能下降：标准化 WIPS 接近 1。唯一的例外是会话 3，其性能下降是由原始实验中极其繁忙的服务器引起的。

接下来，我们仔细观察图 9 中的一个会话，并在图 10 中显示其性能如何随时间变化。图中的点显示每秒的 WIPS。两条曲线显示了 TPC-W 计算出的 30 秒窗口内的移动平均值。我们在图中标记了实时迁移的开始和结束时间。启用自膨胀后，迁移期间传输的内存量约为 600 MB。该图验证了实时迁移不会导致明显的性能下降。迁移持续时间不到 10 秒。回想一下，我们的算法每 10 分钟调用一次。

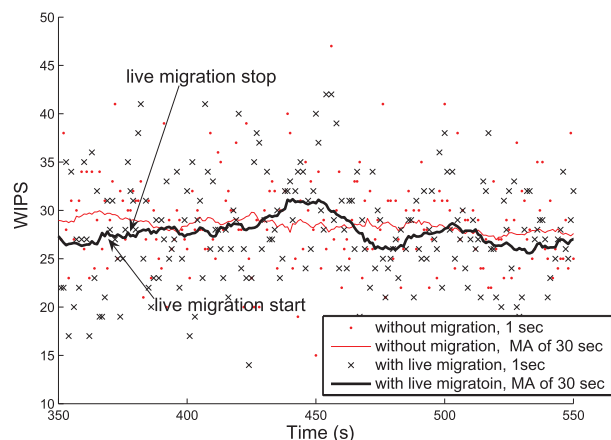


图 10. 有和没有实时迁移的 TPC-W 性能。

6.3 资源平衡

回想一下，调度算法的目标是将具有不同资源需求的工作负载混合在一起，从而提高服务器容量的整体利用率。在此实验中，我们将了解我们的算法如何处理 CPU、内存和网络密集型工作负载的混合。我们像以前一样改变 CPU 负载。我们通过向虚拟机发送一系列网络数据包来注入网络负载。内存密集型应用程序是通过按需分配内存来创建的。我们再次从一个由两个 PM 和四个 VM 组成的小规模实验开始，以便我们可以在图 11 中呈现所有服务器的结果。两行代表两个 PM。两列分别代表 CPU 和网络维度。本实验的内存消耗保持在较低水平。

最初，PM₁ 上的两个虚拟机是 CPU 密集型的，而 PM₂ 上的两个虚拟机是网络密集型的。我们逐渐增加他们瓶颈资源的负载。大约 500 秒，由于 PM₂ 中的网络过载，VM₄ 从 PM₂ 迁移到 PM₁。然后大约 600 秒，由于 PM₁ 中的 CPU 过载，VM₁ 从 PM₁ 迁移到 PM₂。现在，系统达到了稳定状态，两个 PM 的资源利用率达到了平衡，每个 PM 都有一个 CPU 密集型虚拟机和一个网络密集型虚拟机。后来我们逐渐减少所有 VM 的负载，使两个 PM 都成为冷点。我们可以看到 PM₁ 上的两个 VM 通过绿色计算整合到 PM₂ 上。

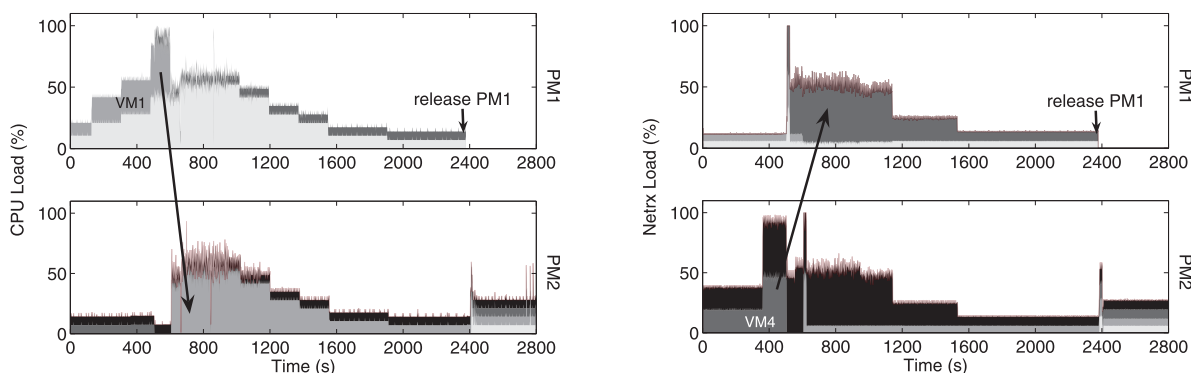


图 11. 混合工作负载的资源平衡。

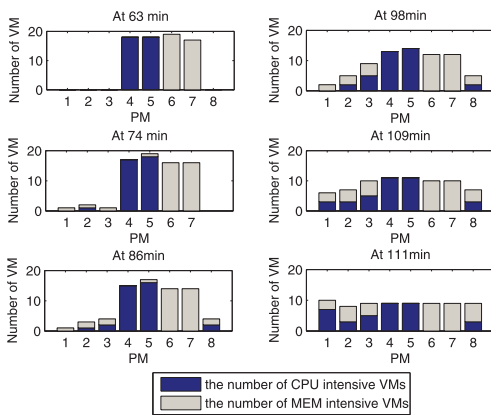


图 12. VM随时间的分布。

接下来，我们将实验规模扩展到运行超过 8 个 PM 的一组 72 个虚拟机。一半的虚拟机是 CPU 密集型的，而另一半是内存密集型的。最初，我们将所有虚拟机的负载保持在较低水平，并将所有 CPU 密集型虚拟机部署在 PM₄ 和 PM₅ 上，同时将所有内存密集型虚拟机部署在 PM₆ 和 PM₇ 上。然后我们逐渐增加所有 VM 上的负载，使底层 PM 成为热点。图 12 显示了算法如何随着时间的推移将 VM 传播到其他 PM。从图中我们可以看出，该算法适当地平衡了两种类型的 VM。该图还显示，当我们增加负载时，PM 组上的负载变得非常平衡。

7 RELATED WORK

7.1 应用层的资源分配

之前在[14]和[15]中针对数据中心环境研究了 Web 应用程序的自动扩展。在 MUSE [14] 中，每个服务器都有系统中运行的所有 Web 应用程序的副本。前端 L7 交换机中的调度算法可确保合理地服务请求，同时最大限度地减少未充分利用的服务器数量。工作[15]使用网络流算法在其运行实例之间分配应用程序的负载。对于面向连接的 Internet 服务（例如 Windows Live Messenger），工作 [10] 提出了一种用于负载调度和服务器配置的集成方法。上述所有工作都不使用虚拟机，并且要求应用程序构建在多层架构中，并通过前端调度程序提供负载平衡。相比之下，我们的工作针对的是 Amazon EC2 风格的环境，在该环境中，它对应用程序在虚拟机内构建的内容和方式没有限制。虚拟机被视为黑匣子。资源管理仅在整个虚拟机的粒度上完成。

MapReduce [16] 是另一种流行的云服务，其中数据局部性是其性能的关键。Quincy[17]在任务调度中采用最小成本流模型来最大化数据局部性，同时保持不同作业之间的公平性。“延迟调度”算法[18]用执行时间换取数据局部性。工作[19]为作业和用户分配动态优先级以促进资源分配。

7.2 虚拟机实时迁移的资源分配

VM 实时迁移是虚拟化环境中广泛使用的动态资源分配技术[8]、[12]、

[20]。我们的工作也属于这一类。Sandpiper 将多维负载信息组合成单个体积指标 [8]。它根据 PM 的卷以及每个 PM 中的虚拟机的卷大小比 (VSR) 对 PM 列表进行排序。不幸的是，这抽象了做出迁移决策时所需的关键信息。然后，它会按预先排序的顺序考虑 PM 和 VM。我们在补充文件的第 1 节中给出了一个具体示例，该文件可在线获取，其中他们的算法在过载期间选择了错误的虚拟机进行迁移，并且无法缓解热点问题。我们还在实际实验中比较了我们的算法和他们的算法。补充文件的第 5 节对结果进行了分析（可在线获取），以显示它们的行为有何不同。此外，他们的工作不支持绿色计算，并且在负载预测等许多其他方面与我们的工作不同。

HARMONY 系统跨多个资源层应用虚拟化技术[20]。它使用虚拟机和数据迁移来缓解服务器上、网络设备和存储节点上的热点问题。它引入了扩展向量积 (EVP) 作为资源利用不平衡的指标。他们的负载平衡算法是针对多维背包问题的 Toyoda 方法 [21] 的变体。与我们的系统不同，他们的系统不支持绿色计算，负载预测留作未来的工作。在在线提供的补充文件的第 6 节中，我们分析了 VectorDot 的行为与我们的工作不同的现象，并指出了我们的算法可以更好地利用剩余资源的原因。

[12] 研究了动态放置虚拟服务器以最大限度地减少 SLA 违规。他们将其建模为装箱问题，并使用众所周知的首次拟合近似算法定期计算 VM 到 PM 布局。然而，该算法主要是为离线使用而设计的。当应用在虚拟机资源需求动态变化的在线环境中时，可能会产生大量的迁移。

7.3 绿色计算

人们已经做出了许多努力来减少数据中心的能源消耗。基于硬件的方法包括用于降低冷却功率的新颖热设计，或采用与功率成比例的低功率硬件。工作[22]使用动态电压和频率缩放 (DVFS) 来根据其负载调整 CPU 功率。正如补充文件第 7 节所述，我们不使用 DVFS 进行绿色计算。PowerNap[23]借助固态硬盘 (SSD) 和自刷新 DRAM 等新硬件技术来实现全运行和低功耗状态之间的快速转换（小于 1ms），从而可以在较短的空闲间隔内“小憩”。当服务器进入睡眠状态时，Somniloquy [24] 通知驻留在特殊设计的 NIC 上的嵌入式系统委托主操作系统。它给人一种服务器始终处于活动状态的错觉。

我们的工作属于纯软件低成本解决方案的类别[10]、[12]、[14]、[25]、[26]、[27]。与 Somniloquy [24] 类似，SleepServer [26] 在专用服务器上作为委托启动虚拟机，而不是依赖于特殊的 NIC。LiteGreen [25] 不使用委托。相反，它会迁移桌面操作系统，以便桌面可以休眠。它要求桌面通过共享存储进行虚拟化。

Jettison [27] 发明了“部分虚拟机迁移”，这是实时虚拟机迁移的一种变体，它只迁移必要的工作集，同时留下不常用的数据。

8 CONCLUSION

我们介绍了云计算服务资源管理系统的设计、实现和评估。我们的系统根据不断变化的需求自适应地将虚拟资源复用到物理资源。我们利用偏度指标将具有不同资源特性的虚拟机进行适当组合，从而充分利用服务器的能力。我们的算法为具有多资源约束的系统实现了过载避免和绿色计算。

致谢

作者要感谢匿名审稿人的宝贵反馈。This work was supported by the National Natural Science Foundation of China (Grant No. 61170056), the National High Technology Research and Development Program (“863” Program) of China (Grant No. 2013AA013203), National Basic Research Program of China (Grant No. 2009CB320505) and Digital Resource Security Protection Service Based on Trusted Identity Federation and Cloud Computation SubProject of 2011 Information Security Special 国家发展和改革委员会资助项目。

参考

[1] M. Armbrust 等人, “云之上: 伯克利云计算视角”, 技术报告, 大学。加利福尼亚州伯克利, 2009 年 2 月。[2] L. Siegle, “让它崛起: 企业 IT 特别报告”, 《经济学家》, 卷. 389, 第 3-16 页, 2008 年 10 月。[3] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt 和 A. Warfield, “Xen 和虚拟化艺术”, Proc. ACM 症状。操作系统原理 (SOSP '03), 2003 年 10 月。[4] “Amazon 弹性计算云 (Amazon EC2)”, <http://aws.amazon.com/ec2/>, 2012。[5] C. Clark, K. Fraser, S. Hand, J.G. Hansen, E. Jul, C. Limpach, I. Pratt 和 A. Warfield, “虚拟机实时迁移”, Proc. 症状。网络系统设计和实现 (NSDI '05), 2005 年 5 月。[6] M. Nelson, B.-H. Lim 和 G. Hutchins, “虚拟机的快速透明迁移”, Proc. USENIX Ann. 技术会议, 2005 年。[7] M. McNett, D. Gupta, A. Vahdat 和 G.M. Voelker, “Usher: 管理虚拟机集群的可扩展框架”, Proc. 大型安装系统管理会议 (LISA '07), 2007 年 11 月。[8] T. Wood, P. Shenoy, A. Venkataramani 和 M. Yousef, “虚拟机迁移的黑盒和灰盒策略”, Proc. 症状。网络系统设计和实现 (NSDI '07), 2007 年 4 月。[9] C.A. Waldspurger, “VMware ESX Server 中的内存资源管理”, Proc. 症状。操作系统设计和实现 (OSDI '02), 2002 年 8 月。[10] G. Chen, H. Wenbo, J. Liu, S. Nath, L. Rigas, L. Xiao 和 F. Zhu, “连接密集型 Internet 服务的能源感知服务器配置和负载分派”, Proc. USENIX 症状。网络系统设计和实现 (NSDI '08), 2008 年 4 月。[11] P. Padala, K.-Y. Hou, K.G. Shin, X. Zhu, M. Uysal, Z. Wang, S. Singhal 和 A. Merchant, “多个虚拟化资源的自动控制”, Proc. ACM European conf. 计算机系统 (EuroSys '09), 2009 年。

[12] N. Bobroff, A. Kochut 和 K. Beatty, “用于管理 SLA 违规的虚拟机动态放置”, Proc. IFIP/IEEE 国际研讨会。集成网络管理 (IM '07), 2007。[13] “TPC-W: 事务处理性能委员会”, <http://www.tpc.org/tpcw/>, 2012。[14] J.S. 蔡斯, D.C. 安德森, P.N. 塔卡尔, A.M. Vahdat 和 R.P. Doyle, “管理托管中心的能源和服务器资源”, Proc. ACM 症状。操作系统原理 (SOSP '01), 2001 年 10 月。[15] C. Tang, M. Steinder, M. Spreitzer 和 G. Pacifici, “企业数据中心的可扩展应用程序放置控制器”, Proc. 国际万维网会议 (WWW '07), 2007 年 5 月。[16] M. Zaharia, A. Konwinski, A.D. Joseph, R.H. Katz 和 I. Stoica, “提高异构环境中的 MapReduce 性能”, Proc. 症状。操作系统设计和实现 (OSDI '08), 2008。[17] M. Isard, V. Prabhakaran, J. Currey, U. Wieder, K. Talwar 和 A. Goldberg, “昆西: 分布式计算集群的公平调度”, Proc. ACM 症状。操作系统原理 (SOSP '09), 2009 年 10 月。[18] M. Zaharia, D. Borthakur, J. Sen Sarma, K. Elmeleegy, S. Shenker 和 I. Stoica, “延迟调度: 在集群调度中实现局部性和公平性的简单技术”, Proc. 欧洲会议计算机系统 (EuroSys '10), 2010。[19] T. Sandholm 和 K. Lai, “使用调节动态优先级进行 Mapreduce 优化”, Proc. 国际联合会议计算机系统的测量和建模 (SIGMETRICS '09), 2009。[20] A. Singh, M. Korupolu 和 D. Mohapatra, “服务器存储虚拟化: 数据中心的集成和负载均衡”, Proc. ACM/IEEE 会议超级计算, 2008。[21] Y. Toyoda, “获得零一编程问题近似解的简化算法”, 管理科学, 卷. 21, 第 1417-1427 页, 1975 年 8 月。[22] R. Nathuji 和 K. Schwan, “Virtualpower: 虚拟化企业系统中的协调电源管理”, Proc. ACM SIGOPS 症状。操作系统原理 (SOSP '07), 2007。[23] D. Meisner, B.T. 黄金和 T.F. Wenis, “PowerNap: 消除服务器空闲电源”, Proc. 国际会议编程语言和操作系统的架构支持 (ASPLOS '09), 2009。[24] Y. Agarwal, S. Hodges, R. Chandra, J. Scott, P. Bahl 和 R. Gupta, “Somniloquy: 增强网络接口以减少 PC 能源使用”, Proc. USENIX 症状。网络系统设计和实现 (NSDI '09), 2009。[25] T. Das, P. Padala, V.N. Padmanabhan, R. Ramjee 和 K.G. Shin, “Litegreen: 使用虚拟化在网络桌面中节省能源”, Proc. USENIX 安. 技术会议, 2010。[26] Y. Agarwal, S. Savage 和 R. Gupta, “Sleepserver: 减少企业环境中 PCS 能耗的纯软件方法”, Proc. USENIX 安. 技术会议, 2010。[27] N. Bila, E.d. 劳拉, K. 乔希, H.A. Lagar-Cavilla, M. Hiltunen 和 M. Satyanarayanan, “Jettison: 通过部分虚拟机迁移进行高效空闲桌面整合”, Proc. ACM 欧洲会议计算机系统 (EuroSys '12), 2012 年。



肖震, 北京大学计算机系教授。2001 年 1 月获得康奈尔大学博士学位, 后在新泽西州 AT&T 实验室担任高级技术人员, 后在 IBM T.J. 担任研究人员。沃森研究中心。他的研究兴趣包括云计算、虚拟化和各种分布式系统问题。他是 IEEE 和 ACM 的高级会员。



宋伟嘉在北京理工大学获得学士和硕士学位。目前，他是北京大学的博士生。他目前的研究重点是云系统中的资源调度问题。



陈琪于2010年获得北京大学学士学位，目前是北京大学博士生。她目前的研究重点是云计算和并行计算。

。有关此或任何其他计算主题的更多信息，请访问我们的数字图书馆：
www.computer.org/publications/dlib。